

Learning to Optimize Network Routing

Approximating Optimization-Based Routing Decisions Using Deep Neural Networks and Graph Neural Networks

Saurav Mittal

Centre for Development of Advanced Computing (C-DAC)

July 15, 2026

Project Objective

- Network routing can be formulated as an optimization problem in which incoming traffic flows are assigned paths according to the current state of the network.
- Optimization-based routing can generate high-quality routing decisions, but repeatedly solving the optimization problem can become computationally expensive.
- The objective of this work is to generate routing decisions using an optimization framework and train machine-learning models to approximate these decisions.
- We compare:
 - a conventional Deep Neural Network (DNN), and
 - a Message-Passing Graph Neural Network (GNN).

End-to-End Workflow

1. Traffic Data

GÉANT Traffic Matrices



2. Flow Generation

Generate Individual Traffic Flows



3. Routing Optimization

Process Flows Sequentially and Select Routes



4. Supervised Learning

Construct dataset containing the SD pairs, network state and optimized path and
Train DNN/GNN Models

GÉANT Network and Traffic Data

- The experiments use traffic data associated with the **GÉANT research network**.
- Network topology used in this work:
 - 23 nodes,
 - 74 directed links.
- Traffic demand is represented using **origin–destination traffic matrices**.
- Each traffic-matrix element describes aggregate demand between one source and one destination during a measurement interval.



Traffic Matrix Does Not Mean Link Load

Consider a traffic-matrix element

$$T_{10,6} = 100 \text{ Mbps.}$$

This means:

Origin–Destination Demand

There is an aggregate traffic demand of 100 Mbps **originating at node 10 and terminating at node 6.**

It does **not** mean that 100 Mbps is directly carried by a physical link between nodes 10 and 6.

For example, the traffic may be routed as

$$10 \rightarrow 4 \rightarrow 6$$

continue...

or

$$10 \rightarrow 7 \rightarrow 3 \rightarrow 6.$$

But, not when demand is between other links.

$$11 \rightarrow 10 \rightarrow 7 \rightarrow 3 \rightarrow 6 \rightarrow 8.$$

Important Distinction

The traffic matrix defines the demand. The actual load on each network link is determined only after routing paths are assigned.

From Traffic Matrices to Individual Flows

Traffic matrices provide aggregate demand, whereas the routing simulation operates on individual traffic flows.

Each generated flow is represented as

$$f = (s, d, t_{\text{arrival}}, \tau, b),$$

where

- s = source node,
- d = destination node,
- t_{arrival} = time of arrival (TOA),
- τ = flow duration,
- b = bandwidth demand.

The generated flows are stored in

`flow.parquet`.

From Traffic Matrices to Individual Flows

For each traffic matrix, the aggregate demand TM_{ij} between source i and destination j is converted into individual flows.

- 1 Consider every source–destination pair (i, j) .
- 2 Allocate approximately $N = 1000$ flows across all OD pairs in proportion to their traffic demand:

$$K_{ij} \approx N \frac{TM_{ij}}{\sum_{i,j} TM_{ij}}$$

- 3 K_{ij} represents the number of individual flows generated for OD pair (i, j) .

Traffic Matrix

Aggregate OD demand TM_{ij}



Flow Allocation

Demand between OD pairs.



Individual Flows

K_{ij} flows for each OD pair (i, j)

Generating the Flow Event Sequence

For each of the K_{ij} flows between source i and destination j , the flow characteristics are generated.

- Arrival time within the 15-minute traffic-matrix interval:

$$TOA \sim U(T_{TM}, T_{TM} + 900)$$

- Flow duration:

$$\tau \sim \mathcal{N}(60, 12)$$

Generated Flow

Source s
Destination d
Arrival time t_{arrival} Duration τ
Bandwidth b



Combine All Flows

Flows from all traffic matrices



Global Event Sequence

`flow.parquet` sorted by arrival
time

continue...

- Flow traffic volume:

$$v_k \sim \mathcal{N}\left(\frac{TM_{ij}}{K_{ij}}, \frac{TM_{ij}}{3K_{ij}}\right), \quad k = 1, \dots, K_{ij},$$

with the generated flow volumes scaled so that their total matches the aggregate OD traffic demand TM_{ij} .

- Flow bandwidth is then computed as

$$b_k = \frac{v_k}{\tau_k},$$

where v_k is the traffic volume assigned to flow k and τ_k is its duration.

Sequential Event-Driven Routing

The generated flows are processed in chronological order while maintaining the set of currently active flows.

For each new incoming flow:

- 1 Read the next event:

$$f = (s, d, t_{\text{arrival}}, \tau, b)$$

- 2 Inspect the currently active flows.
- 3 Remove every active flow satisfying:

$$t_{\text{departure}} \leq t_{\text{arrival}}$$

- 4 Release the bandwidth previously occupied by each departing flow.

Next Flow Arrival

$$(s, d, TOA, \tau, b)$$



Check Active Flows

Which flows have
already expired?



Process Departures

Remove expired flows

Routing the New Flow and Updating the Network

After all departures preceding the new arrival have been processed, the current network state is used to route the incoming flow.

- 1 And recompute the utilization of all 74 directed links.
- 2 Generate candidate paths from source s to destination d .
- 3 Select a feasible path according to the routing objective and current network state.
- 4 Update the utilization of every link on the selected path.
- 5 Store the network state and routing action as a supervised-learning example.

Current Network State

74-link utilization
vector



Routing Decision

Generate candidate paths and
select a feasible route



Update and Store

Dynamic Network State

For each directed link e , the current utilization is

$$u_e = \frac{\sum_{f \in \mathcal{A}_e} b_f}{C_e},$$

where

- $\mathcal{A}_e$ is the set of active flows using link e ,
- b_f is the bandwidth of flow f ,
- C_e is the link capacity.

The complete network state is represented by

$$\mathbf{u} = [u_1, u_2, \dots, u_{74}].$$

Key Point

The routing decision for a new flow depends on the utilization created by all currently

Arrival and Departure Events

The dataset contains both arrival and departure events.

For a flow using a set of links $\mathcal{P}$:

$$y_e = \begin{cases} +1, & e \in \mathcal{P} \quad \text{for an arriving flow,} \\ -1, & e \in \mathcal{P} \quad \text{for a departing flow,} \\ 0, & e \notin \mathcal{P}. \end{cases}$$

Thus the target is a 74-dimensional vector:

$$\mathbf{y} = [y_1, y_2, \dots, y_{74}], \quad y_e \in \{-1, 0, +1\}.$$

- +1: bandwidth is added to the link,
- -1: bandwidth is removed from the link,
- 0: the link is unaffected by the event.

Routing Optimization

For an incoming flow from source s to destination d :

- ① Generate candidate simple paths

$$\mathcal{P}_{s,d} = \{P_1, P_2, \dots, P_K\}.$$

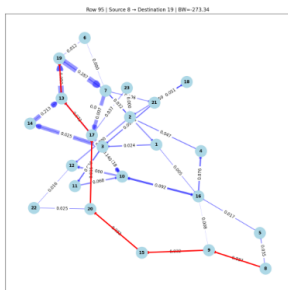
- ② Select exactly one candidate path.
- ③ Ensure that the selected route is feasible under the current network state.
- ④ Prefer routes containing less-congested links.

The implemented path cost is

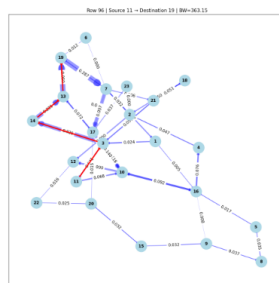
$$C(P) = \sum_{e \in P} (u_e + \lambda),$$

where u_e penalizes congested links and the small per-edge penalty λ discourages unnecessarily long routes.

Example of an Optimization Decision



(a) Flow Departure



(b) Flow Arrival

The network is shown with its current link utilizations. For each source–destination pair and bandwidth demand, the optimization procedure evaluates feasible candidate routes and selects a path according to the current network state. The highlighted route represents the selected path.

Output of the Routing Pipeline

For every routing event, the following information is stored:

Information	Representation
Source node	<code>source_node</code>
Destination node	<code>destination_node</code>
Arrival time	<code>TOA</code>
Duration	<code>Duration</code>
Bandwidth	<code>Bandwidth</code>
Selected path	<code>Path</code>
Target	<code>Link_1 ... Link_74</code>
Network state	<code>Utilization_1 ... Utilization_74</code>

Chunk-Based Processing

The complete flow dataset is processed in smaller sequential chunks.



- Flows remain in temporal order.
- The utilization vector at the end of one chunk is carried forward to the next chunk.
- This preserves continuity of the simulated network state across chunk boundaries.

Important

Chunking is used for data handling; it does not reset the network state.

Data-Pipeline Pitfall: Missing Destination Category

During preprocessing, source and destination nodes are one-hot encoded. However, one-hot encoding creates columns only for categories that are actually present in the selected data.

In the first selected subset of flows:

```
count(destination_node = 10) = 0.
```

Therefore,

```
destination_node_10.0
```

was not automatically created.

Consequence

The dataset contained 22 destination columns instead of the expected 23, shifting subsequent hard-coded positional indices.

Learning Routing Policies from Optimization

The optimization framework generates supervised examples of the form

Network State $\longrightarrow$ Routing Decision.

For each event, the model receives:

- the source node,
- the destination node,
- the current utilization of all 74 links,
- the flow bandwidth demand.

The model predicts

$$\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{74}],$$

s representing the Target.

Machine-Learning Input Representation

The flattened model input consists of:

$$23 + 23 + 74 + 1 = 121$$

features.

Feature Block	Dimension
Source node one-hot vector	23
Destination node one-hot vector	23
Current edge utilizations	74
Bandwidth demand	1
Total	121

The prediction target contains one value for each of the 74 directed network links.

DNN Baseline

The DNN treats the complete network state as a flattened feature vector.

$$121 \rightarrow 2048 \rightarrow 1024 \rightarrow 512 \rightarrow 74.$$

- Input: complete 121-dimensional network state.
- Hidden representation learned using fully connected layers.
- Output: routing action for all 74 directed links.
- Number of trainable parameters:

≈ 2.91 million.

Limitation

The DNN receives the network as a flat vector and does not explicitly use the graph connectivity.

Why Use a Graph Neural Network?

A communication network is naturally represented as a graph

$$G = (V, E).$$

Routing decisions depend directly on:

- which nodes are connected,
- the utilization of neighboring links,
- alternative routes through the topology,
- the relationship between source and destination.

A Graph Neural Network can therefore learn directly from the network structure.

Message Passing

Each node updates its representation by aggregating information from its neighbors. Stacking multiple message-passing layers allows information to propagate over progressively larger neighborhoods.

Proposed Message-Passing GNN

The GNN uses three message-passing layers:

$$46 \rightarrow 128 \rightarrow 128 \rightarrow 128.$$

- **Node features:** source and destination indicators.
- **Edge features:** current utilization and flow bandwidth information.
- **Three NNConv layers:** propagate information through the network topology.
- **Global mean pooling:** creates a graph-level representation of the current network state.
- **Edge-level MLP:** predicts the routing action for each directed link.

DNN and GNN Architecture Comparison

TABLE I
COMPARISON BETWEEN THE PROPOSED GNN AND DNN MODELS.

Property	GNN	DNN
Input Representation	23 source and 23 destination indicators 74 edge utilizations 1 bandwidth feature	121-dimensional flattened feature vector
Architecture	Node features: 46(source+destination), Edge features: 2 (current utilization and bandwidth demand) 3 NNConv message-passing layers (46→128→128→128) (3-hop message propagation) Residual connection: $h_1 + h_3$ Global mean pooling (128-D network-state representation) Edge representation: source node (128) + destination node (128) + network state (128) + edge features (2) = 386-D MLP: 386→256→128→1	FC: 121→2048 FC: 2048→1024 FC: 1024→512 FC: 512→74
Trainable Parameters	4.81M	2.91M
Validation MAE	0.027	0.040

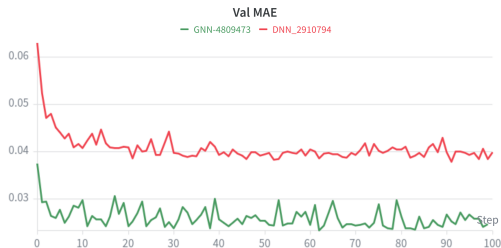
Model Comparison

Property	GNN	DNN
Input representation	Graph structured	121-D flat vector
Topology explicitly used	Yes	No
Message passing	3 NNConv layers	None
Trainable parameters	4.81 M	2.91 M
Validation MAE	0.027	0.040

The topology-aware GNN achieves a lower validation error than the conventional DNN baseline.

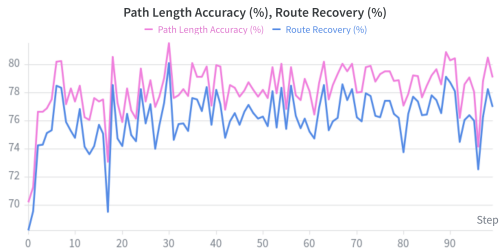
Model Performance

Training Performance



The GNN converges to a lower validation MAE (**0.027**) than the DNN baseline (**0.040**), an improvement of approximately **30%**.

Routing Accuracy



The GNN recovers approximately **75%** of complete MILP routes and predicts the correct path length in approximately **80%** of cases.

References

-  S. Uhlig, B. Quoitin, J. Lepropre, and S. Balon, “Providing Public Intradomain Traffic Matrices to the Research Community,” *ACM SIGCOMM Computer Communication Review*, vol. 36, no. 1, pp. 83–86, 2006.
-  J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural Message Passing for Quantum Chemistry,” *Proceedings of ICML*, 2017.
-  K. Rusek et al., “RouteNet: Leveraging Graph Neural Networks for Network Modeling and Optimization in SDN,” *IEEE Journal on Selected Areas in Communications*, 2020.
-  Y. Bengio, A. Lodi, and A. Prouvost, “Machine Learning for Combinatorial Optimization: A Methodological Tour d’Horizon,” *European Journal of Operational Research*, 2021.

Thank You